

CineSeat Rehberi

CQRS, Onion Architecture ve Code First

— Projemize Özel —

Hiçbir terim bilindiği varsayılmadan, tamamen CineSeat'in kendi gerçek kodundan alınan örneklerle açıklanmıştır.

Ekip: Kaan Sezer · Alptuğ Dursun · Ömer Faruk Çendek · İzzettin Berke Kuş
Ağustos 2026

İçindekiler

Bölüm 0 — Bu Rehber Nasıl Okunur	5
Ortak metaforumuz: Sinema gişesi	5
Neden bu kadar uğraşıyoruz?	5
Bölüm 1 — Temel Kavramlar	7
1.1 CRUD	7
1.2 Ön bilgi: Assembly, CLR ve ORM	7
1.3 POCO (Plain Old CLR Object)	8
1.4 DTO (Data Transfer Object)	8
1.5 Mapping (Eşleme)	9
1.6 Validator (Doğrulayıcı)	10
1.7 Use Case (Kullanım Senaryosu) ve Screaming Architecture	11
1.8 Web API	11
Bölüm 2 — Veritabanı Tarafı: Code First	13
2.1 Neden Code First?	13
2.2 DbContext	13
2.3 Fluent API	14
2.4 Composite Key'in kuzeni: Bileşik Unique Index	14
2.5 Migration	15
2.6 SaveChanges'i özelleştirmek: AuditableEntityInterceptor	15
2.7 CI/CD	16
Bölüm 3 — Katmanlar ve Aralarındaki İletişim	17
3.1 Onion Architecture	17
3.2 Dependency Inversion Principle (DIP)	17
3.3 Dependency Injection (DI)	18
3.4 WebAPI'nin referans problemi ve çözümü	19
Bölüm 4 — CQRS ve Handler Dünyası	21
4.1 CQS'ten CQRS'e	21
4.2 MediatR	21
4.3 Handler'ın anatomisi	21

4.4 Query handler ve projection	22
4.5 Command/Query neden record değil?	23
4.6 Pipeline Behavior — CineSeat'in iki halkası	23
4.7 Notification (Domain Event) — CineSeat'te henüz yok	24
Bölüm 5 — Bir İsteğin Uçtan Uca Yolculuğu: Rezervasyon Oluşturma	25
Bölüm 6 — Tasarım Deseni Sözlüğü (CineSeat merceğinden)	28
6.1 Repository Pattern	28
6.2 Unit of Work	28
6.3 Specification Pattern	28
6.4 Result Pattern	28
6.5 Diğer desenler (CineSeat'te görülenler)	29
Bölüm 7 — Mimari Yaklaşımlar: CineSeat Neden Onion + CQRS?	30
Bölüm 8 — CineSeat Proje İskeleti	31
Referans yönü	31
Bölüm 9 — Ne Zaman Ne Kullanmalı?	33
Sağlıklı ilerleme sırası — CineSeat için somut sonraki adımlar	33
Bölüm 10 (Bonus) — Frontend Entegrasyonu: React ↔ ASP.NET Core	34
10.1 CORS: farklı origin, farklı dünya	34
10.2 apiClient.js — merkezi fetch katmanı	34
10.3 Rol eşleme: Admin/User ↔ admin/member	35
Ek A — Terimler Sözlüğü	36
Ek B — CineSeat Kod Tabanından Gözlemler	38
İyi uygulanmış noktalar	38
Somut iyileştirme fırsatları	38

Bölüm 0 — Bu Rehber Nasıl Okunur

Bu rehber, CineSeat'i inşa ederken kullandığımız CQRS, Onion Architecture ve Code First kavramlarını, hiçbir terimi bildiğimiz varsayılmadan, **kendi gerçek kodumuzdan** örneklerle anlatır. Ekip dışından biri (ör. bir jüri üyesi, bir direktör) bu belgeyi okuduğunda hem "bu terim ne demek" sorusuna hem de "biz bunu nerede, nasıl yaptık" sorusuna cevap bulabilmeli.

Üç şey birlikte gider:

- **Gerçek hayat benzetmesi** — kavramın neye benzediği
- **Problem** — bu kavram olmasaydı başımıza ne gelirdi
- **CineSeat kodu** — projemizde pratikte nasıl görüldüğü (dosya adı ve satırlarla)

Ortak metaforumuz: Sinema gişesi

Baştan sona aynı benzetmeyi kullanacağız — üstelik bu sefer uydurma bir örnek değil, tam olarak CineSeat'in kendi iş alanı. Bir sinemanın gişesi de, yazılımdaki katmanlar da aynı ilkeyle çalışır: **herkesin tek bir işi vardır ve kimse başkasının işine karışmaz.**

Gişede	CineSeat'te	Görevi
Müşteri	Kullanıcı (tarayıcı / mobil)	İstekte bulunur
Gişe vitrini, afiş	API sözleşmesi (Controller route'ları)	Nelerin istenebileceğini söyler
Gişe görevlisi	Controller (CinemasController, ReservationsController...)	İsteği alır, ilgili yere iletir, sonucu döner
Sipariş fişi	Command / Query (CreateReservationCommand...)	Ne istendiğini yazan kâğıt
Vardiya amiri	MediatR (ISender / IMediator)	Fişi okur, doğru uzmana verir
Uzman personel	Handler (CreateReservationCommandHandler...)	İşlemi baştan sona yürütür
Salon / koltuk kuralları	Domain (Entity + iş kuralı kontrolleri)	"Aynı koltuk iki kez satılamaz"
Depo görevlisi	Repository (IReadRepository<T> / IWriteRepository<T>)	Veriyi veritabanından getirir, yazar
Bilet arşivi	Veritabanı (PostgreSQL)	Verinin saklandığı yer
Gün sonu kasa sayımı	SaveChangesAsync()	Değişiklikleri tek seferde işler
Bilet	DTO (MovieDto, ReservationDto...)	Müşteriye gösterilecek, sunuma hazır hâl

Bu tabloya rehber boyunca defalarca döneceğiz. Şimdilik sadece şu fikri aklında tut: gişedeki görevli salona çıkıp koltuk göstermez, salon sorumlusu da gişede bilet kesmez. Katmanlı mimarinin bütün amacı budur.

Neden bu kadar uğraşıyoruz?

Dürüst cevap: yazılım projeleri zamanla ölür. Ölüm sebebi genelde şudur — bir şeyi değiştirmek istersin, ama o şey başka on yere dokunmuştur. Veritabanını PostgreSQL'den başka bir şeye taşımak istersin, rezervasyon kuralların çöker. JWT yerine başka bir kimlik doğrulama yöntemi eklemek istersin, controller'ların yarısını açman gerekir. Bir hata bulursun, düzeltirsin, üç yeni hata çıkar.

Bu rehberdeki her kavram, tek bir soruya verilmiş cevaptır:

Kilit soru

Bir şeyi değiştirdiğimde başka neyi değiştirmek zorunda kalacağım? İyi mimari bu sayıyı küçültür. Kalan her şey detaydır.

CineSeat'te bunun somut karşılığı şu: `CreateReservationCommandHandler` içindeki koltuk-kitleme mantığını değiştirdiğimizde `MoviesController`'a, `AuthService`'e ya da veritabanı bağlantı dizesine dokunmamız gerekmiyor. Bu rehber, bunun nasıl sağlandığını baştan sona anlatıyor.

Bölüm 1 — Temel Kavramlar

1.1 CRUD

Açılımı Create, Read, Update, Delete — Türkçesi: Oluştur, Oku, Güncelle, Sil. Terim, mimari kararlarında bir sınıflandırma aracı olarak kullanılır.

- **"Bu bir CRUD modülü"** demek → içinde ciddi iş kuralı yok, veri sadece tabloya yazılıp okunuyor. CineSeat'te City, District, Genre, Technology modülleri tam olarak budur: CreateCityCommandHandler sadece bir City nesnesi oluşturup kaydeder, başka kontrol yoktur.
- **"Bu CRUD değil"** demek → verinin değişmesi bir sürü kurala tabi. CineSeat'te CreateReservationCommandHandler CRUD değildir: koltuk seansın salonuna mı ait, koltuk zaten satılmış mı, başka biri tarafından kilitli mi, kampanya aktif mi, sepet kampanyanın asgari tutarını geçiyor mu — sekiz ayrı kontrolden geçmeden tek satır veritabanına yazılmaz.

Neden önemli?

Mimari kararını buna göre verirsın. Saf CRUD ekranları için (Genres, Technologies) ağır bir doğrulama/iş kuralı katmanı kurmak zaman kaybıdır — nitekim CineSeat'te bu modüllerin çoğunun validator'ı yok. Karmaşık modüllerde (Reservations, Comments) ise tam tersi geçerli. Aynı projede modüllere farklı davranmak tamamen meşrudur.

1.2 Ön bilgi: Assembly, CLR ve ORM

CLR (Common Language Runtime). C# kodu doğrudan işlemcide çalışmaz; önce IL'e (Intermediate Language) derlenir, sonra CLR bunu çalışma anında makine koduna çevirip yürütür. Bellek yönetimi (çöp toplayıcı), tip güvenliği ve istisna yönetimi CLR'nin sağladıklarıdır.

Assembly. Derlenmiş bir .NET projesinin çıktısı — bir .dll veya .exe. CineSeat tam olarak **5 projeden**, dolayısıyla 5 assembly'den oluşuyor:

CineSeat.Domain.dll	<- Core/CineSeat.Domain
CineSeat.Application.dll	<- Core/CineSeat.Application
CineSeat.Persistence.dll	<- Infrastructure/CineSeat.Persistence
CineSeat.Infrastructure.dll	<- Infrastructure/CineSeat.Infrastructure
CineSeat.WebAPI.exe	<- Presentation/CineSeat.WebAPI (giriş noktası)

CineSeat.sln içindeki 5 proje

Bu ayrımın gerçek gücü .csproj dosyalarındaki ProjectReference etiketlerinde görülür. CineSeat.Application.csproj şunu içerir:

```
<ItemGroup>
  <ProjectReference Include="..\CineSeat.Domain\CineSeat.Domain.csproj" />
</ItemGroup>

<ItemGroup>
  <PackageReference Include="FluentValidation.DependencyInjectionExtensions" ... />
  <PackageReference Include="MediatR" ... />
  <!--
    Microsoft.EntityFrameworkCore BİLİNÇLİ OLARAK YOK.
    Application katmanı hangi ORM/veritabanı kullanıldığını bilmemelidir.
  -->
</ItemGroup>
```

Yani CineSeat.Application sadece CineSeat.Domain'i referans alıyor; Microsoft.EntityFrameworkCore paketi bilerek eklenmemiş. Bu, sadece bir tavsiye değil — Application katmanındaki bir dosyada DbContext kullanmaya kalksan derleyici seni durdurur, çünkü o assembly EF Core'u hiç görmüyor. Bölüm 3'te bunun nasıl aşıldığını (IAsyncQueryExecutor) göreceğiz.

ORM (Object-Relational Mapper). Veritabanı satır/kolonlardan, C# ise nesnelerden oluşur; bu uyumsuzluğa *impedance mismatch* denir. ORM ikisi arasında tercümanlık yapar. CineSeat'te bu iş **Entity Framework Core** (Npgsql sağlayıcısıyla PostgreSQL'e karşı) tarafından yapılıyor.

1.3 POCO (Plain Old CLR Object)

Tanım: hiçbir kütüphaneye bulaşmamış, sade bir C# sınıfı. CineSeat'in Movie entity'si tam bir POCO örneğidir — System.* dışında hiçbir bağımlılığı yok, EF Core özneliği ([Table], [Key] vb.) da taşıyor:

```
public class Movie : BaseEntity
{
    public string Title { get; set; }
    public short Duration { get; set; }
    public string Description { get; set; }
    public short AgeLimit { get; set; }
    public string Language { get; set; }
    public string Poster { get; set; }
    public DateTime StartDate { get; set; }
    public DateTime EndDate { get; set; }
    public decimal AvgScore { get; set; }

    public ICollection<UserFavorite> UserFavorites { get; set; }
    public ICollection<Comment> Comments { get; set; }
    public ICollection<MovieGenre> MovieGenres { get; set; }
    public ICollection<Showtime> Showtimes { get; set; }
}
```

backend/Core/CineSeat.Domain/Entities/Movie.cs (gerçek kod)

Movie tabloların nasıl kurulacağını, hangi veritabanını kullandığımızı bilmiyor. Bu bilgi, göreceğimiz gibi, ApplicationDbContext.OnModelCreating içinde — yani Persistence katmanında — yaşıyor.

Dikkat

POCO olmak "hiç property'si olmayan" demek değildir. DateTime, string, ICollection<T> gibi System.* altındaki temel tipleri kullanmak POCO'luğu bozmaz. Bozan şey, dışarıdan gelen bir NuGet paketine (EF Core, bir üçüncü parti kütüphane) bağımlı olmaktır.

1.4 DTO (Data Transfer Object)

Sadece veri taşımak için var olan, davranışsız sınıf. İki ayrı problemi çözer.

Problem 1: Güvenlik

CineSeat'in User entity'si parola hash'i ve tuzunu taşır:


```
public class User : BaseEntity
{
    public string Email { get; set; }
    public string PasswordHash { get; set; } // ASLA dışarı çıkmamalı
    public string PasswordSalt { get; set; } // ASLA dışarı çıkmamalı
    public long RoleId { get; set; }
    ...
}
```

backend/Core/CineSeat.Domain/Entities/User.cs (özet)

LoginCommandHandler bu entity'yi hiçbir zaman doğrudan döndürmez; onun yerine AuthUserDto üretir — sadece Id, Name, Surname, Username, Email, Role alanlarını taşıyan, hash/salt içermeyen ayrı bir sınıf:

```
public class AuthUserDto
{
    public long Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public string Surname { get; set; } = string.Empty;
    public string Username { get; set; } = string.Empty;
    public string Email { get; set; } = string.Empty;
    public string Role { get; set; } = string.Empty;
}
```

backend/Core/CineSeat.Application/Features/Auth/DTOs/AuthUserDto.cs (gerçek kod)

Problem 2: Kırılganlık ve fazla veri

GetMoviesQueryHandler de aynı mantıkla Movie entity'sini değil MovieDto'yu döndürür — hem PasswordHash türü bir sızıntı riski yoktur burada, ama film listesi ekranı için Showtimes, Comments, UserFavorites gibi ilişkili koleksiyonları da çekmek anlamsız ve yavaş olurdu. Select(...) projeksiyonu sadece ihtiyaç duyulan 9 alanı SQL'e taşır (bkz. 1.5 ve 4.4).

Gerçek hayat karşılığı — Gişe

Gişe görevlisi sana tezgâhın arkasındaki bilet ruloları kutusunu uzatmaz; sana sadece kestiği bileti verir. User = kutu, AuthUserDto = bilet. Müşterinin kutuda kaç bilet kaldığını görmesine gerek yoktur.

1.5 Mapping (Eşleme)

Entity'den DTO'ya veri kopyalama işi. CineSeat'te **AutoMapper** ya da **Mapster** kullanılmıyor — tüm eşlemeler elle, ya LINQ Select projeksiyonu içinde ya da handler'ın son satırında yeni bir DTO oluşturularak yapılıyor:

```
// GetMoviesQueryHandler içinde -- doğrudan sorgu projeksiyonu
.Select(m => new MovieDto
{
    Id = m.Id,
    Title = m.Title,
    Duration = m.Duration,
    Description = m.Description,
    AgeLimit = m.AgeLimit,
    Language = m.Language,
    Poster = m.Poster,
    StartDate = m.StartDate,
    EndDate = m.EndDate,
    AvgScore = m.AvgScore
});
```

backend/.../Movies/Queries/GetMovies/GetMoviesQueryHandler.cs (gerçek kod)

Neden elle?

Otomatik eşleme kütüphanesi 30 sınıfın olduğu büyük bir projede zaman kazandırır, ama elle yazılan eşlemede bir alanı unutursan **derleyici** hata verir; otomatik eşlemede unutulmuş alan sessizce null kalır. CineSeat 19 entity ile hâlâ "küçük-orta" ölçekte sayılır — elle yazmak burada bilinçli ve makul bir tercih; kütüphane, gerçekten acı vermeye başladığında eklenebilir.

1.6 Validator (Doğrulayıcı)

Gelen verinin **şeklen** doğru olup olmadığını kontrol eden sınıf. CineSeat **FluentValidation** kullanıyor; her validator, ValidationBehavior pipeline halkası tarafından otomatik olarak (bkz. 4.6) handler çalışmadan önce tetikleniyor:

```
public class CreateCinemaCommandValidator : AbstractValidator<CreateCinemaCommand>
{
    public CreateCinemaCommandValidator()
    {
        RuleFor(x => x.Name)
            .NotEmpty().WithMessage("Sinema adı boş olamaz.")
            .MaxLength(150).WithMessage("Sinema adı en fazla 150 karakter olabilir.");

        RuleFor(x => x.Address)
            .NotEmpty().WithMessage("Adres boş olamaz.");

        RuleFor(x => x.Latitude)
            .InclusiveBetween(-90m, 90m).WithMessage("Enlem -90 ile 90 arasında olmalıdır.");

        RuleFor(x => x.Longitude)
            .InclusiveBetween(-180m, 180m).WithMessage("Boylam -180 ile 180 arasında olmalıdır.");

        RuleFor(x => x.DistrictId)
            .GreaterThan(0).WithMessage("Geçerli bir ilçe seçilmelidir.");
    }
}
```

backend/.../Cinemas/Commands/CreateCinema/CreateCinemaCommandValidator.cs (gerçek kod)

Kritik ayırım: nerede ne kontrol edilir? CineSeat'in kendi kodundan somut örneklerle:

Soru	Nerede?	CineSeat'ten örnek
Enlem -90/+90 arasında mı?	Validator	CreateCinemaCommandValidator — şekil kontrolü, veritabanı gerekmez
İlçe gerçekten var mı?	Handler	CreateCinemaCommandHandler: _districtRead.GetByIdAsync(...) ile veritabanına bakılır
Kampanya aktif mi / sepet eşiği geçildi mi?	Handler	CreateReservationCommandHandler.CalculateDiscountAsync — bağlam bilgisi gerekir
Koltuk zaten satılmış mı?	Handler	Aynı handler — ITicketReadRepository sorgusu
Bu kullanıcı bu filme daha önce yorum yapmış mı?	Handler	AddCommentCommandHandler — veri durumuna bağlı

Gözlem

CineSeat'te entity'ler *anemic* (davranışsız, sadece veri) olduğu için üçüncü katman — "entity'nin kendi içi" — orijinal rehberde önerilen kadar kullanılmıyor. İş kuralları neredeyse tamamen handler'larda toplanmış. Bunun ne anlama geldiğini Bölüm 3.1'de (Anemic vs Rich Model) ayrıca ele alıyoruz.

1.7 Use Case (Kullanım Senaryosu) ve Screaming Architecture

Sistemin dışarıya sunduğu, kullanıcının yapmak istediği tek bir anlamlı iş. CineSeat'in gerçek use case'lerinden bazıları: **Rezervasyon oluştur, Yorum ekle, Film ara, Giriş yap, Favorilere ekle, Koltuk kilitle.**

Robert C. Martin'in *Screaming Architecture* fikri: mimari, uygulamanın NE yaptığını bağırması, hangi framework'le yazıldığını değil. CineSeat'in Application/Features/ klasörü tam olarak bunu uyguluyor — teknik role göre değil, işe göre klasörleniyor:

```
Features/  
  Auth/          Commands/Login, Commands/Register  
  Movies/        Commands/CreateMovie, UpdateMovie, DeleteMovie, Queries/GetMovies, GetMovieById  
  Cinemas/       Commands/CreateCinema, UpdateCinema, DeleteCinema, Queries/GetCinemasByCity,  
                  GetCinemaById  
  Reservations/  Commands/CreateReservation, CancelReservation, Queries/GetMyReservations,  
                  GetReservationById  
  Comments/      Commands/AddComment, DeleteComment, Queries/GetCommentsByMovie  
  Showtimes/, Seats/, SeatLocks/, Tickets/, Campaigns/, Genres/, ...
```

backend/Core/CineSeat.Application/Features/ (gerçek klasör ağacı, kısaltılmış)

"Rezervasyon iptali nerede?" sorusunun cevabı tek bir yerde:

Features/Reservations/Commands/CancelReservation/. Bu yapı, projede **407 C# dosyasının** neredeyse tamamının nereye ait olduğunu isminden anlamayı sağlıyor.

1.8 Web API

Bir programın dışarıya HTTP üzerinden açtığı kapı. CineSeat'in CinemasController'ı, Bölüm 4'te ayrıntısıyla ele alacağımız MediatR deseninin en yalın hâlini gösteriyor:

```
[ApiController]  
[Route("api/[controller]")]  
public class CinemasController : ControllerBase  
{  
    private readonly IMediator _mediator;  
    public CinemasController(IMediator mediator) => _mediator = mediator;  
  
    [HttpGet]  
    public async Task<IActionResult> GetByCity(  
        [FromQuery] long cityId, [FromQuery] int pageNumber = 1, [FromQuery] int pageSize = 20)  
        => Ok(await _mediator.Send(  
            new GetCinemasByCityQuery { CityId = cityId, PageNumber = pageNumber, PageSize = pageSize }));  
  
    [Authorize(Roles = RoleNames.Admin)]  
    [HttpPost]  
    public async Task<IActionResult> Create([FromBody] CreateCinemaCommand command)  
        => Ok(await _mediator.Send(command));  
}
```

backend/Presentation/CineSeat.WebAPI/Controllers/CinemasController.cs (kısaltılmış, gerçek kod)

Controller'ın içinde tek satır iş mantığı yok — sadece isteği alıyor, MediatR'a devrediyor, sonucu HTTP cevabına çeviriyor. `[Authorize(Roles = RoleNames.Admin)]` satırı, `GetByCity` herkese açıkken `Create`'in sadece Admin rolüne izin verdiğini gösteriyor.

HTTP fiili	CRUD	CineSeat örneği
GET	Read	GET /api/cinemas?cityId=1 — sinema listesi
POST	Create	POST /api/reservations — rezervasyon oluştur
PUT	Update	PUT /api/cinemas/{id} — sinema güncelle

HTTP fiili	CRUD	CineSeat örneđi
DELETE	Delete	DELETE /api/cinemas/{id} — sinema sil

Bölüm 2 — Veritabanı Tarafı: Code First

2.1 Neden Code First?

Veritabanı ile kodun ilişkisini kurmanın üç yolu vardır (Code First, Database First, Model First); aradaki fark "doğrunun kaynağı hangisi" sorusunda düşümlenir. CineSeat **Code First** kullanıyor: önce `Movie`, `Reservation` gibi POCO sınıfları yazılıyor, EF Core tabloları onlardan üretiyor. Kanıtı, `Migrations/` klasöründeki gerçek geçmiş:

```
20260728062258_InitialCreate.cs      <- İlk şema
20260729070947_CompleteDatabaseDesign.cs <- Tüm entity ilişkileri tamamlandı
20260729094346_FixSchemaIssues.cs    <- Şema düzeltmeleri
```

backend/Infrastructure/CineSeat.Persistence/Migrations/ (gerçek dosya adları)

Bu üç isim tek başına Code First'ün en somut faydasını gösteriyor: şemadaki her değişikliğin ne zaman, hangi sebeple yapıldığı Git geçmişinde duruyor. Kimsenin SQL script'i e-postayla göndermesi gerekmedi.

2.2 DbContext

`ApplicationDbContext`, CineSeat'in 19 tablosunu tek yerde toplayan ana sınıf:

```
public class ApplicationDbContext : DbContext
{
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options) : base(options) { }

    public DbSet<Role> Roles { get; set; }
    public DbSet<User> Users { get; set; }
    public DbSet<Movie> Movies { get; set; }
    public DbSet<Cinema> Cinemas { get; set; }
    public DbSet<Reservation> Reservations { get; set; }
    public DbSet<Ticket> Tickets { get; set; }
    // ... toplam 19 DbSet

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);
        // ... Fluent API kuralları (bkz. 2.3) burada
        ApplySoftDeleteFilter(modelBuilder);
    }
}
```

backend/Infrastructure/CineSeat.Persistence/Data/ApplicationDbContext.cs (özet)

Kayıt şu şekilde yapılıyor — dikkat edilmesi gereken iki nokta var: `UseSnakeCaseNamingConvention()` (PostgreSQL dünyasında `movie_genres`, `created_at` gibi küçük-harf-alt-çizgi isimlendirme âdettir) ve `AddInterceptors(...)` (bkz. 2.6):

```
services.AddDbContext<ApplicationDbContext>((serviceProvider, options) =>
    options.UseNpgsql(configuration.GetConnectionString("DefaultConnection"))
        .UseSnakeCaseNamingConvention()
        .AddInterceptors(serviceProvider.GetRequiredService<AuditableEntityInterceptor>()));
```

backend/Infrastructure/CineSeat.Persistence/ServiceRegistration.cs (gerçek kod)

Change Tracker — en sihirli kısım

`_context.Movies.FirstAsync(...)` ile çektiğin bir `Movie` nesnesinin property'sini değiştirip `SaveChangesAsync()` çağırdığında hiçbir `UPDATE` cümlesi yazmamış olursun; EF Core nesnenin ilk hâlini hafızada tutar, kaydetme anında farkı bulur ve sadece değişen kolonlar için SQL üretir. `AddCommentCommandHandler`'da `movie.AvgScore = ...` satırından sonra `_movieWrite.Update(movie)` çağrılması tam olarak bunu tetikler.

Okuma sorgularında bu takip bedavaya gelmez — her nesne için bir kopya tutulur. `CineSeat`'in tüm `Read` repository metotları bu yüzden tracking parametresi alır ve query handler'lar tutarlı biçimde tracking: `false` geçer (bkz. 4.4).

2.3 Fluent API

`CineSeat`'te tablo kuralları (`[Table]`, `[Required]` gibi) Data Annotation **değil**, Fluent API ile tanımlanıyor — ama orijinal önerinin aksine, ayrı `IEntityTypeConfiguration<T>` dosyalarına da bölünmemiş; tamamı doğrudan `OnModelCreating` içinde:

```
modelBuilder.Entity<UserFavorite>()
    .HasIndex(uf => new { uf.UserId, uf.MovieId })
    .IsUnique();

modelBuilder.Entity<User>()
    .HasIndex(u => u.Email)
    .IsUnique();

modelBuilder.Entity<Movie>()
    .Property(m => m.AvgScore)
    .HasPrecision(3, 2);

modelBuilder.Entity<Reservation>()
    .Property(r => r.Total)
    .HasPrecision(10, 2);
```

backend/.../Data/ApplicationDbContext.cs — OnModelCreating (gerçek kod, kısaltılmış)

CineSeat'te gerçekte böyle

Rehberin "ideal" önerisi her entity için ayrı bir `IEntityTypeConfiguration<T>` sınıfı (bkz. `Configurations/` klasörü) yazmaktır — büyük projelerde `OnModelCreating`'in yüzlerce satıra şişmesini engeller. `CineSeat`'te 19 entity için bu henüz gerekli görülmemiş; tüm kurallar tek dosyada, yönetilebilir boyutta (~140 satır) duruyor. Entity sayısı arttıkça bölünmesi gündeme gelebilir — bu bilinçli bir borç, bir hata değil.

2.4 Composite Key'in kuzeni: Bileşik Unique Index

Composite key (birden fazla kolonun birlikte birincil anahtar olması), `CineSeat`'te klasik biçimde kullanılmıyor — junction (bağlantı) tabloları kendi `Id`'sine sahip. Ama aynı problemi ("bu ikili birden fazla kez tekrarlanamaz") çözen kuzeni, **bileşik unique index**, üç yerde bilerek kullanılıyor:

Entity	Bileşik index	Engellediği durum
UserFavorite	(UserId, MovieId)	Aynı kullanıcı aynı filmi iki kez favoriye ekleyemez
MovieGenre	(MovieId, GenreId)	Aynı türe bir filmi iki kez atayamazsın
SeatLock	(ShowtimeId, SeatId)	Aynı seans+koltuk aynı anda iki kez kilitlenemez

SeatLock örneği özellikle kritik: iki kullanıcı aynı koltuğu aynı anda seçtiğinde uygulama kodu yarışı kaçırma bile **veritabanı** ikinci kilit satırını reddeder. İş kuralı en alt seviyede, veritabanı tarafından garanti edilmiş olur.

2.5 Migration

```
# Yeni migration oluştur
dotnet ef migrations add FixSchemaIssues -p CineSeat.Persistence -s CineSeat.WebAPI

# Veritabanına uygula (CineSeat'te bunu Program.cs başlangıçta otomatik yapıyor,
# bkz. DbInitializer.SeedAsync -> context.Database.MigrateAsync())
dotnet ef database update -p CineSeat.Persistence -s CineSeat.WebAPI
```

Gerçek proje adlarıyla komutlar

-p migration dosyalarının yazılacağı proje (DbContext orada), -s uygulamanın başlangıç projesi (appsettings.json'daki bağlantı dizesi oradan okunur).

Dikkat — üretimde Database.Migrate()

CineSeat şu an context.Database.MigrateAsync()’i doğrudan Program.cs içinde, uygulama her başladığında çalıştırıyor. Geliştirme ortamında pratik olsa da, uygulama birden fazla kopya hâlinde (ör. yük dengeleyici arkasında) çalıştırıldığında bu yaklaşım risklidir — iki kopya aynı anda şemayı değiştirmeye çalışabilir. Üretime geçişte migration’ın CI/CD hattında, dağıtımdan önce, tek seferde çalıştırılması daha güvenlidir.

2.6 SaveChanges'i özelleştirmek: AuditableEntityInterceptor

CineSeat, orijinal rehberin önerdiği "SaveChangesAsync'i override et" yönteminden **daha modern** bir EF Core aracı kullanıyor: SaveChangesInterceptor. Mantık aynı, ama DbContext'in kendisi kirlenmiyor:

```
public class AuditableEntityInterceptor : SaveChangesInterceptor
{
    public override InterceptionResult<int> SavingChanges(
        DbContextEventData eventData, InterceptionResult<int> result)
    {
        ApplyAuditFields(eventData.Context);
        return base.SavingChanges(eventData, result);
    }

    private static void ApplyAuditFields(DbContext? context)
    {
        var now = DateTimeOffset.UtcNow;
        foreach (var entry in context.ChangeTracker.Entries<BaseEntity>())
        {
            if (entry.State == EntityState.Added)
                entry.Entity.CreatedAt = now;
            else if (entry.State == EntityState.Modified)
                entry.Entity.UpdatedAt = now;
        }
    }
}
```

backend/.../Data/Interceptors/AuditableEntityInterceptor.cs (gerçek kod)

Bu sayede hiçbir handler CreatedAt = DateTime.UtcNow yazmak zorunda kalmıyor — 407 dosyanın hiçbirinde bu satırı bulamazsın, çünkü gerek yok.

Bulgu — IsDeleted alanı henüz yazma tarafına bağlı değil

BaseEntity.IsDeleted var ve ApplicationDbContext.ApplySoftDeleteFilter her entity için WHERE is_deleted = false global filtresini otomatik ekliyor — yani **okuma tarafı** soft-delete'e hazır. Ama kod tabanında IsDeleted = true atayan **tek bir satır bile yok**: WriteRepository<T>.Remove() hâlâ Table.Remove(model) çağırıp gerçek DELETE üretiyor (EntityState.Deleted). Yani bugün bir kaydı silmek hâlâ kalıcı siliniyor; soft-delete altyapısı kurulu ama devreye alınmamış. Bu, Ek B'de tekrar ele alınan somut bir iyileştirme fırsatı.

2.7 CI/CD

CI (Continuous Integration) ve CD (Continuous Delivery/Deployment), her kod gönderiminde otomatik derleme/test ve dağıtım yapılmasıdır — birleştirme kâbuslarını (merge hell) önler.

CineSeat'in şu anki durumu

Depoda .github/workflows/ klasörü yok; CI/CD hattı henüz kurulmamış, derleme ve (var olduğunda) testler geliştiricinin kendi makinesinde elle çalıştırılıyor. Bu, ekibin şu an nerede durduğunu dürüstçe göstermek için burada belirtiliyor — Bölüm 9'da "sağlıklı ilerleme sırası" içinde bir sonraki adım olarak öneriliyor.

Kurulduğunda nasıl görüneceği (GitHub Actions örneği):

```
name: CI
on:
  push:
    branches: [ main ]
  pull_request:

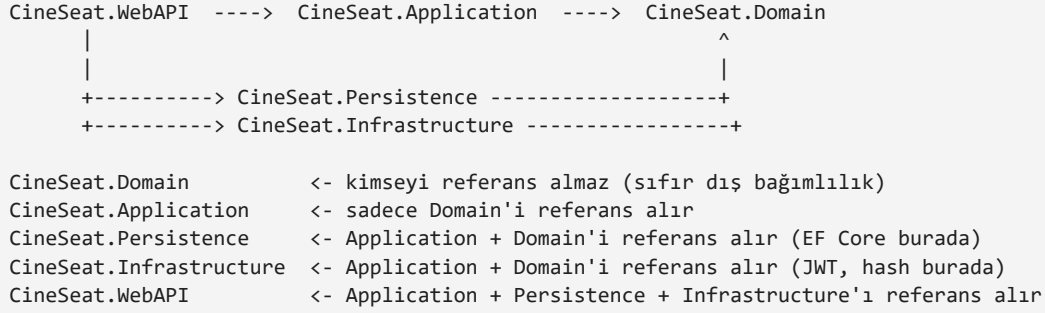
jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-dotnet@v4
        with: { dotnet-version: '10.0.x' }
      - run: dotnet restore backend/CineSeat.sln
      - run: dotnet build backend/CineSeat.sln --no-restore -c Release
      - run: dotnet test backend/CineSeat.sln --no-build -c Release
```

Örnek — henüz uygulanmadı

Bölüm 3 — Katmanlar ve Aralarındaki İletişim

3.1 Onion Architecture

2008'de Jeffrey Palermo'nun ortaya attığı fikir: bağımlılıklar sadece içe doğru akmalı. Dış katman iç katmanı bilir, iç katman dışarıyı asla bilmez. CineSeat'in 5 projesi bu kuralı fiziksel olarak (derleyici tarafından zorlanan ProjectReference'larla) uyguluyor:



backend/CineSeat.sln — gerçek referans yönleri

Katmanların içeriği, CineSeat'te somut olarak:

Katman	CineSeat'te ne var
Domain	19 entity (Movie, Reservation, Ticket...), BaseEntity, enum'lar (ReservationStatus, TicketType...). Hiçbir NuGet paketi yok.
Application	Features/ altında ~40 use case (Command+Handler+Validator+DTO), repository arayüzleri , IAsyncQueryExecutor, pipeline behavior'lar
Persistence	ApplicationDbContext, repository implementasyonları , EfAsyncQueryExecutor, AuditableEntityInterceptor, migration'lar
Infrastructure	JwtTokenService, Pbkdf2PasswordHasher — dış dünyaya bakan servisler
WebAPI (Presentation)	18 controller, ExceptionHandlingMiddleware, CurrentUserService, Program.cs

Anemic model — bilinçli bir tercih

Orijinal literatür "rich domain model"i (entity'nin kendi kurallarını kendi metotlarıyla koruması) önerir. CineSeat'in entity'leri bunun tersi: **anemic** — `Movie.AvgScore`, `Reservation.Status` gibi tüm property'ler `public get; set;`, hiçbir iş kuralı içermiyor. Kural kontrolü (kampanya aktif mi, koltuk müsait mi, kullanıcı zaten yorum yapmış mı) tamamen handler'larda toplanmış (Bölüm 1.6, 4.3, 5).

Bu bir hata mı?

Hayır — orijinal rehberin kendi ilkesiyle örtüşüyor: "kural varsa entity'nin içinde olsun, kural yoksa entity'yi zorla zenginleştirme." CineSeat'in kuralları (kampanya hesabı, koltuk çakışması) genelde **birden fazla entity'yi aynı anda** ilgilendiriyor (Reservation + Ticket + SeatLock + Campaign) — bu tip çapraz kontroller zaten doğal olarak handler seviyesinde yaşar. Tek bir riski var: aynı kuralı ileride ikinci bir handler'dan (ör. bir arka plan görevinden) tetiklemek gerekirse, kuralı kopyalamamak için dikkat gerekir.

3.2 Dependency Inversion Principle (DIP)

Arayüz iç katmanda tanımlanır, implementasyon dış katmanda yaşar. CineSeat'te bu, her repository çifti için birebir uygulanmış:

```
// Application/Repositories/IReadRepository.cs (İÇ KATMAN – sadece sözleşme)
public interface IReadRepository<T> : IRepository<T> where T : BaseEntity
{
    IQueryable<T> GetAll(bool tracking = true);
    IQueryable<T> GetWhere(Expression<Func<T, bool>> method, bool tracking = true);
    Task<T?> GetByIdAsync(long id, bool tracking = true, CancellationToken ct = default);
}

// Persistence/Repositories/ReadRepository.cs (DIŞ KATMAN – implementasyon)
public class ReadRepository<T> : IReadRepository<T> where T : BaseEntity
{
    private readonly ApplicationDbContext _context;
    protected DbSet<T> Table => _context.Set<T>(); // DbSet DIŞARI AÇILMIYOR

    public IQueryable<T> GetAll(bool tracking = true)
    {
        var query = Table.AsQueryable();
        if (!tracking) query = query.AsNoTracking();
        return query;
    }
    // ...
}
```

backend/Core/.../Repositories/IReadRepository.cs + backend/Infrastructure/.../ReadRepository.cs (gerçek kod)

Her entity için ince bir alt sınıf bu genel implementasyonu miras alıyor — örneğin CityReadRepository sadece üç satır:

```
public class CityReadRepository : ReadRepository<City>, ICityReadRepository
{
    public CityReadRepository(ApplicationDbContext context) : base(context) { }
}
```

backend/Infrastructure/CineSeat.Persistence/Repositories/City/CityReadRepository.cs (tam kod)

IAsyncQueryExecutor — CineSeat'e özgü, orijinal rehberde olmayan bir çözüm

IReadRepository<T>.GetAll() bir IQueryable<T> döndürür — ama ToListAsync, CountAsync, AnyAsync, FirstOrDefaultAsync metotları LINQ'in değil, **Microsoft.EntityFrameworkCore** paketinin extension metotlarıdır. Handler'lar bunları doğrudan çağırıyorsa Application katmanı EF Core'a bağımlı hâle gelirdi (bkz. 1.2 — o paket bilerek eklenmemiştir). CineSeat bu boşluğu kendi arayüzüyle kapatmış:

IAsyncQueryExecutor (Application'da tanımlı, sadece 4 metot) → EfAsyncQueryExecutor (Persistence'ta, EF Core'a delege eder). Handler "sorguyu asenkron çalıştır" der, "EF ile çalıştır" demez. Bu, DIP'in orijinal rehberdeki IEmailService örneğiyle birebir aynı desenin, sorgu materyalizasyonuna uygulanmış hâli.

```
public interface IAsyncQueryExecutor
{
    Task<List<T>> ToListAsync<T>(IQueryable<T> query, CancellationToken ct = default);
    Task<int> CountAsync<T>(IQueryable<T> query, CancellationToken ct = default);
    Task<bool> AnyAsync<T>(IQueryable<T> query, CancellationToken ct = default);
    Task<T?> FirstOrDefaultAsync<T>(IQueryable<T> query, CancellationToken ct = default);
}
```

backend/Core/CineSeat.Application/Common/Interfaces/IAsyncQueryExecutor.cs (tam kod)

3.3 Dependency Injection (DI)

DIP bir prensiptir, DI onu uygulayan tekniktir. Handler'lar hangi somut sınıfın enjekte edileceğine kendileri karar vermez — bu bilgi kayıt anında (uygulama açılışında) IServiceCollection'a yazılır. CineSeat'in Persistence

katmanı, **40 repository**'yi (20 entity × Read+Write) tek dosyada topluca kaydediyor:

```
public static void AddPersistenceServices(this IServiceCollection services, IConfiguration
    configuration)
{
    services.AddSingleton<AuditableEntityInterceptor>();
    services.AddDbContext<ApplicationDbContext>((sp, options) => ...);
    services.AddSingleton<IAsyncQueryExecutor, EfAsyncQueryExecutor>();

    services.AddScoped<ICityReadRepository, CityReadRepository>();
    services.AddScoped<ICityWriteRepository, CityWriteRepository>();
    services.AddScoped<IMovieReadRepository, MovieReadRepository>();
    services.AddScoped<IMovieWriteRepository, MovieWriteRepository>();
    // ... toplam 40 satır, her entity için Read+Write
}
```

backend/Infrastructure/CineSeat.Persistence/ServiceRegistration.cs (kısaltılmış, gerçek kod)

Lifetime	Ne zaman yeni nesne?	CineSeat'te kim kullanıyor
AddSingleton	Bir kez, uygulama boyunca	AuditableEntityInterceptor, IAsyncQueryExecutor, ITokenService, IPasswordHasher
AddScoped	Her HTTP isteği için bir kez	ApplicationDbContext, tüm 40 repository, ICurrentUserService
AddTransient	Her istendiğinde yeni	İki pipeline behavior (LoggingBehavior, ValidationBehavior)

Dikkat — Captive Dependency

Bir Singleton servisin içine Scoped bir servis enjekte edilemez (ör. AuditableEntityInterceptor Singleton olduğu için ApplicationDbContext'i constructor'ından ALMAZ — SavingChanges çağrısında kendisine parametre olarak gelen DbContext'i kullanır). Bu ayrım CineSeat'in kodunda doğru uygulanmış; aksi hâlde .NET geliştirme ortamında Cannot consume scoped service from singleton hatasıyla açılışa patlardı.

3.4 WebAPI'nin referans problemi ve çözümü

Kural: WebAPI sadece Application'ı bilmeli, Persistence'teki somut sınıfları (CityReadRepository gibi) görmemeli. Ama DI kaydını bir yerde yapmak gerekiyor. Çözüm, her katmanın **kendi kaydını kendi extension metoduyla** yapması — CineSeat'in Program.cs'i tam olarak bunu yapıyor:

```
var builder = WebApplication.CreateBuilder(args);

// Her katman kendi servislerini kendi kaydeder — Program.cs hiçbirinin
// içeriğini bilmiyor, sadece "hazır ol" diyor.
builder.Services.AddApplicationServices();
builder.Services.AddPersistenceServices(builder.Configuration);
builder.Services.AddInfrastructureServices(builder.Configuration);

builder.Services.AddHttpContextAccessor();
builder.Services.AddScoped<ICurrentUserService, CurrentUserService>();
```

backend/Presentation/CineSeat.WebAPI/Program.cs (gerçek kod, başlangıç kısmı)

Program.cs içinde CityReadRepository kelimesi hiç geçmiyor. WebAPI, CineSeat.Persistence.csproj'u teknik olarak hâlâ referans alıyor (yoksa AddPersistenceServices() metodunu çağıramazdı), ama tek bir metotla, tek bir noktadan temas ediyor. Bu tek temas noktasına *composition root* (birleştirme kökü) denir.

Gerçek hayat karşılığı — Gişe

Sinema müdürü sabah gelir, "gişe, gişeni hazırla; mutfak/büfe, kendini hazırla" der ve gider. Hangi kasanın hangi çekmeceye konulacağını, patlamış mısır makinesinin nasıl temizleneceğini bilmez, bilmesine de gerek yoktur. `AddPersistenceServices()`, `AddInfrastructureServices()` çağrıları tam olarak bu "hazır ol" emridir.

Bölüm 4 — CQRS ve Handler Dünyası

4.1 CQS'ten CQRS'e

Bertrand Meyer'in CQS (Command Query Separation) ilkesi: bir metot ya durumu değiştirir (command) ya veri döndürür (query), ikisini birden yapmaz. Greg Young bunu mimari seviyeye taşıdı: CQRS — okuma ve yazma modelleri tamamen ayrı olsun.

CineSeat'te bu ayrım klasör seviyesinde bile görünür: her Features/X/ altında Commands/ ve Queries/ iki ayrı alt klasördür ve birbirinin sınıflarını hiç kullanmaz. Movies modülünde CreateMovieCommand (yazar, geriye sadece long Id döner) ile GetMoviesQuery (Okur, PagedResult<MovieDto> döner) aynı Movie tablosuna gider ama uçtan uca farklı sınıflar, farklı handler'lardır.

Yaygın yanlış anlama

CQRS "iki ayrı veritabanı" demek değildir. CineSeat tek bir PostgreSQL veritabanı, tek bir ApplicationDbContext kullanıyor. Ayrılan şey **model**dir (yazarken entity, okurken DTO), veri deposu değil.

4.2 MediatR

Mediator deseni: birbiriyle konuşması gereken nesneler doğrudan değil, bir aracı üzerinden haberleşir. CineSeat controller'ları IMediator dışında hiçbir handler'ı, hiçbir repository'yi tanımaz. Kayıt, Application katmanının kendi DependencyInjection.cs'inde, reflection ile otomatik:

```
public static IServiceCollection AddApplicationServices(this IServiceCollection services)
{
    var assembly = Assembly.GetExecutingAssembly();

    // Tüm IRequestHandler<, > implementasyonlarını otomatik bulur ve kaydeder.
    services.AddMediatR(cfg => cfg.RegisterServicesFromAssembly(assembly));

    // Tüm AbstractValidator<T> implementasyonlarını otomatik bulur ve kaydeder.
    services.AddValidatorsFromAssembly(assembly);

    // Pipeline halkaları -- istek yukarıdan aşağıya sırayla geçer.
    services.AddTransient(typeof(IPipelineBehavior<, >), typeof(LoggingBehavior<, >));
    services.AddTransient(typeof(IPipelineBehavior<, >), typeof(ValidationBehavior<, >));

    return services;
}
```

backend/Core/CineSeat.Application/DependencyInjection.cs (tam kod)

CineSeat controller'ları IMediator'ı enjekte ediyor (bazı controller'lar ISender yerine doğrudan IMediator kullanıyor — CineSeat şu an sadece Send çağırdığı için ikisi de eşdeğer çalışıyor, ISender Interface Segregation açısından biraz daha dar bir seçim olurdu).

4.3 Handler'ın anatomisi

Handler'ın tek sorumluluğu orkestrasyon. CreateCinemaCommandHandler, sağlıklı bir CineSeat handler'ının tam boyu — 20 satır, üç adım: **kontrol et → oluştur → kaydet**:

```

public class CreateCinemaCommandHandler : IRequestHandler<CreateCinemaCommand, long>
{
    private readonly ICinemaWriteRepository _write;
    private readonly IDistrictReadRepository _districtRead;

    public async Task<long> Handle(CreateCinemaCommand request, CancellationToken ct)
    {
        // 1) KONTROL ET -- validator yapamazdı, veritabanına bakmak gerekti
        var district = await _districtRead.GetByIdAsync(request.DistrictId, tracking: false, ct);
        if (district is null)
            throw new NotFoundException("İlçe", request.DistrictId);

        // 2) OLUŞTUR
        var cinema = new Cinema
        {
            Name = request.Name, Address = request.Address,
            Latitude = request.Latitude, Longitude = request.Longitude,
            DistrictId = request.DistrictId
        };

        // 3) KAYDET
        await _write.AddAsync(cinema, ct);
        await _write.SaveAsync(ct);
        return cinema.Id;
    }
}

```

backend/.../Cinemas/Commands/CreateCinema/CreateCinemaCommandHandler.cs (gerçek kod)

Handler'ın **yapmadığı** işler ve kimin yaptığı: format doğrulaması → CreateCinemaCommandValidator (pipeline'da handler'dan önce), hata → HTTP kodu çevirisi → ExceptionHandlingMiddleware (Bölüm 5'te), loglama → LoggingBehavior. NotFoundException fırlatmak dışında handler'da tek satır "teknik" kod yok.

4.4 Query handler ve projection

GetCinemasByCityQueryHandler, sayfalama + filtre + projeksiyonun üçünü birden gösteriyor:

```

public async Task<PagedResult<CinemaDto>> Handle(GetCinemasByCityQuery request, CancellationToken ct)
{
    // Sinema -> İlçe -> Şehir: navigation üzerinden filtre (EF SQL'e JOIN olarak çevirir)
    var baseQuery = _read.GetWhere(c => c.District.CityId == request.CityId, tracking: false);

    var totalCount = await _executor.CountAsync(baseQuery, ct);

    var pageQuery = baseQuery
        .OrderBy(c => c.Name)
        .Skip((request.PageNumber - 1) * request.PageSize)
        .Take(request.PageSize)
        .Select(c => new CinemaDto { Id = c.Id, Name = c.Name, Address = c.Address, ... });

    var items = await _executor.ToListAsync(pageQuery, ct);
    return new PagedResult<CinemaDto>(items, totalCount, request.PageNumber, request.PageSize);
}

```

backend/.../GetCinemasByCity/GetCinemasByCityQueryHandler.cs (gerçek kod, kısaltılmış)

Select(...) içinde DTO oluşturmaya *projection* denir — EF Core bunu görünce SQL'e sadece o kolonları koyar; önce ToListAsync() deyip sonra C# tarafında dönüştürmek (bütün kolonları, bütün satırları çekmek) CineSeat'in **hiçbir** query handler'ında yapılmıyor; hepsi Select projeksiyonunu SQL'e taşıyor.

Neden query tarafında repository sınırlı kullanılıyor?

Dikkat edersen GetCinemasByCityQueryHandler doğrudan ICinemaReadRepository.GetWhere(...) + IAsyncQueryExecutor ikilisini kullanıyor, entity'ye özel yeni bir repository metodu eklemiyor. GetWhere genel LINQ ifadesi kabul ettiği için her yeni ekran, repository'ye dokunmadan kendi filtre/projeksiyon mantığını handler içinde yazabiliyor. Bu, orijinal rehberin uyardığı "generic repository 40 metotlu bir çöplüğe dönüşür" riskini CineSeat'te azaltan asıl mekanizma.

4.5 Command/Query neden record değil?

Orijinal literatür, command/query sınıflarının **immutable** (record) olmasını önerir: pipeline'daki bir halka isteği yanlışlıkla değiştirirse (mutable olsaydı) hata ayıklaması günler sürebilecek sessiz bir tutarsızlık doğar.

CineSeat'te bu tercih **yapılmamış** — tüm command/query'ler sıradan, { get; set; } ile mutable class:

```
public class CreateCinemaCommand : IRequest<long>
{
    public string Name { get; set; } = string.Empty;
    public string Address { get; set; } = string.Empty;
    public decimal Latitude { get; set; }
    public decimal Longitude { get; set; }
    public long DistrictId { get; set; }
}
```

backend/.../CreateCinema/CreateCinemaCommand.cs (tam kod)

Bu bir sorun mu?

Pratikte CineSeat'in pipeline'ında sadece iki halka var (Logging, Validation) ve ikisi de isteği değiştirmiyor, sadece okuyor — bu yüzden mutable olmanın teorik riski bugüne kadar hiçbir hataya yol açmamış. Ayrıca ASP.NET Core'un model binding'i mutable class'larla da sorunsuz çalışıyor ([FromBody] CreateCinemaCommand command — controller'lar bunu böyle alıyor). Yine de büyüyen bir pipeline'da (ör. ileride bir CachingBehavior eklenirse) bu, akılda tutulması gereken bir tasarım borcudur.

4.6 Pipeline Behavior — CineSeat'in iki halkası

Bir isteğin, handler'a varmadan önce sırayla geçtiği istasyonlar zinciri. CineSeat'te kayıtlı sırayla iki halka var:

Logging (en dışta) ve **Validation** (içeride):

```
public class LoggingBehavior<TRequest, TResponse> : IPipelineBehavior<TRequest, TResponse>
    where TRequest : notnull
{
    public async Task<TResponse> Handle(TRequest request, RequestHandlerDelegate<TResponse> next,
        CancellationTokens ct)
    {
        var requestName = typeof(TRequest).Name;
        var stopwatch = Stopwatch.StartNew();
        _logger.LogInformation("[CQRS] {RequestName} başladı", requestName);

        var response = await next();

        stopwatch.Stop();
        _logger.LogInformation("[CQRS] {RequestName} bitti ({Ms} ms)", requestName,
            stopwatch.ElapsedMilliseconds);
        return response;
    }
}
```

backend/Core/CineSeat.Application/Common/Behaviors/LoggingBehavior.cs (tam kod)

```

public class ValidationBehavior<TRequest, TResponse> : IPipelineBehavior<TRequest, TResponse>
    where TRequest : notnull
{
    private readonly IEnumerable<IValidator<TRequest>> _validators;

    public async Task<TResponse> Handle(TRequest request, RequestHandlerDelegate<TResponse> next,
        CancellationToken ct)
    {
        if (!_validators.Any())
            return await next(); // Bu isteğin validator'ı yoksa doğrudan geç

        var context = new ValidationContext<TRequest>(request);
        var results = await Task.WhenAll(_validators.Select(v => v.ValidateAsync(context, ct)));
        var failures = results.SelectMany(r => r.Errors).Where(f => f is not null).ToList();

        if (failures.Count > 0)
            throw new ValidationException(failures); // next() ÇAĞIRILMADI -> handler çalışmaz

        return await next();
    }
}

```

backend/Core/CineSeat.Application/Common/Behaviors/ValidationBehavior.cs (tam kod)

`_validators.Any()` kontrolü önemli: Cities gibi bazı modüllerin henüz validator'ı yok — bu durumda pipeline sessizce handler'a geçiyor, akış bozulmuyor.

Neden bir TransactionBehavior yok?

Orijinal rehberde üçüncü, iç halka olarak bir TransactionBehavior önerilir. CineSeat'te buna **gerek yok**, çünkü her handler zaten tek bir `SaveChangesAsync()` çağrısıyla kaydediyor — ve tek bir `SaveChangesAsync()` çağrısı EF Core'da zaten kendiliğinden atomiktir (tek transaction). `CreateReservationCommandHandler` bunun en iyi kanıtı: Reservation, birden fazla Ticket ve silinen SeatLock satırları — dördü de aynı `ApplicationDbContext`'e ait olduğu için **tek** `_reservationWrite.SaveAsync(ct)` çağrısıyla hep birlikte, tek transaction'da kaydedilir (bkz. Bölüm 5 ve 6.2).

4.7 Notification (Domain Event) — CineSeat'te henüz yok

MediatR'in `INotification/IPublisher` mekanizması (bir olayın sıfır veya birden çok dinleyicisi olabilmesi) CineSeat kod tabanında **kullanılmıyor** — grep sonucu: `INotification` implementasyonu yok. Bugün için sorun değil, çünkü henüz "bir işlem tamamlandığında birden fazla bağımsız yan etki tetiklensin" ihtiyacı yok.

Ne zaman işe yarardı?

Örnek senaryo: rezervasyon tamamlandığında (a) kullanıcıya e-posta gönder, (b) sinema doluluk istatistiğini güncelle, (c) kampanya kullanım sayacını artır. Bugün bu üçü tek bir handler içine yazılıydı, handler şişerdi ve e-posta gönderimi başarısız olursa istatistik güncellemesi de hiç çalışmazdı. `ReservationCreatedEvent` : `INotification` gibi bir olay yayınlamak bu üç işi birbirinden bağımsız, ayrı `INotificationHandler` sınıflarına bölebilirdi.

Bölüm 5 — Bir İsteğin Uçtan Uca Yolculuğu: Rezervasyon Oluşturma

Şimdiye kadar öğrendiğimiz her şeyi CineSeat'in en karmaşık, en çok iş kuralı barındıran gerçek akışında birleştirelim: bir kullanıcının koltuk seçip bilet alması. CreateReservationCommandHandler, 8 ayrı kontrolden geçen, tek transaction'da kaydeden ~240 satırlık gerçek koddur.

1) İstek ve kimlik

POST /api/reservations, [Authorize] ile korunan ReservationsController'a düşer. Kim olduğu **istekten değil, JWT'den** okunur:

```
// Command'de UserId BİLİNÇLİ OLARAK YOK -- rezervasyonu yapan kullanıcı
// ICurrentUserService'ten (JWT'den) okunur. İstekle gelseydi herkes
// başkası adına rezervasyon yapabilirdi.
public class CreateReservationCommand : IRequest<ReservationDto>
{
    public long ShowtimeId { get; set; }
    public long? CampaignId { get; set; }
    public string BuyerFname { get; set; } = string.Empty;
    public List<ReservationSeatItem> Seats { get; set; } = new();
}
```

backend/.../CreateReservation/CreateReservationCommand.cs (gerçek kod, kısaltılmış)

IDOR koruması — bilerek eklenmiş bir güvenlik önlemi

IDOR (Insecure Direct Object Reference), bir isteğin "kimin adına" olduğunu istemciye bırakmaktır — kötü niyetli biri userId alanını değiştirip başkası adına işlem yapabilir. CineSeat bunu tamamen kapatmış: Handle metodunun ilk satırı var userId = _currentUser.GetRequiredUserId(); — bu değer JWT'nin imzalı NameIdentifier claim'inden gelir, hiçbir HTTP alanından değil.

2-4) Varlık ve çakışma kontrolleri (handler içinde, sırayla)

- Token geçerli ama kullanıcı sonradan silinmiş olabilir → _userRead.GetByIdAsync ile doğrulanır, yoksa NotFoundException.
- Seans var mı? → _showtimeRead.GetByIdAsync.
- İstekte aynı koltuk iki kez mi seçilmiş? → seatIds.Distinct().Count() != seatIds.Count ile ValidationException.
- Her koltuk gerçekten bu seansın salonuna mı ait? → seat.HallId != showtime.HallId kontrolü.

5-6) Çakışma: satılmış mı, kilitli mi?

```

var bookedSeatIds = await _executor.ToListAsync(
    _ticketRead.GetWhere(
        t => seatIds.Contains(t.SeatId)
            && t.Reservation.ShowtimeId == request.ShowtimeId
            && t.Reservation.Status != ReservationStatus.Cancelled,
        tracking: false)
    .Select(t => t.SeatId),
    cancellationToken);
if (bookedSeatIds.Count > 0)
throw new ConflictException($"Şu koltuklar zaten rezerve edilmiş: {string.Join(", ",
    bookedSeatIds)}");

var conflictingLocks = await _executor.ToListAsync(
    _seatLockRead.GetWhere(
        sl => sl.ShowtimeId == request.ShowtimeId && seatIds.Contains(sl.SeatId)
            && sl.UserId != userId && sl.LockExpiresAt > now,
        tracking: false)
    .Select(sl => sl.SeatId),
    cancellationToken);
if (conflictingLocks.Count > 0)
throw new ConflictException($"Şu koltuklar başka bir kullanıcı tarafından kilitli: ...");

```

backend/.../CreateReservationCommandHandler.cs (gerçek kod)

İkinci sorgu özellikle ilginç: `sl.UserId != userId` sayesinde **kendi kilidin** seni engellemiyor — koltuğu sen kilitleyip sonra satın alman normal akış, engellenmemesi gerekiyor.

7) Fiyat ve kampanya indirimi

```

var price = Math.Round(showtime.BasePrice * TicketTypeMultiplier(item.TicketType), 2);
// TicketTypeMultiplier: Adult=1.0, Student=0.75, Child=0.5
// (Domain'de resmi bir bilet-tipi fiyat kuralı tanımlı değil; bu bir varsayımdır.)

var discount = await CalculateDiscountAsync(request.CampaignId, subtotal, cancellationToken);
var total = subtotal - discount;

// CalculateDiscountAsync içinde:
if (!campaign.IsActive) throw new ConflictException("Bu kampanya aktif değil.");
if (subtotal < campaign.MinCartTotal) throw new ConflictException("Sepet tutarı yetersiz.");

var raw = campaign.Type switch
{
    CampaignType.Percentage => Math.Round(subtotal * campaign.Value / 100m, 2),
    CampaignType.FixedAmount => campaign.Value,
    _ => 0m
};
return Math.Min(raw, subtotal); // indirim ASLA ara toplamı aşamaz

```

backend/.../CreateReservationCommandHandler.cs (gerçek kod, kısaltılmış)

Küçük ama kritik satır: `Math.Min(raw, subtotal)`

Sabit tutarlı (FixedAmount) bir kampanya, ara toplamdan büyük olabilir (ör. 50 TL'lik kampanya, 30 TL'lik sepete uygulanırsa). Bu satır olmasaydı Total negatife düşebilirdi. Küçük bir satır, ama gerçek bir üretim hatasını önüyor.

8) Tek transaction'da kaydet

```

foreach (var ticket in tickets)
    ticket.Reservation = reservation; // navigation fixup -- EF, Id oluşunca FK'yı otomatik set eder

await _reservationWrite.AddAsync(reservation, cancellationToken);
await _ticketWrite.AddRangeAsync(tickets, cancellationToken);

// Bu koltuklara ait varsa kilit satırlarını da aynı transaction'da temizle.
var locksToClear = await _executor.ToListAsync(
    _seatLockRead.GetWhere(sl => sl.ShowtimeId == request.ShowtimeId && seatIds.Contains(sl.SeatId),
        tracking: true),
    cancellationToken);
if (locksToClear.Count > 0)
    _seatLockWrite.RemoveRange(locksToClear);

// TEK SaveChanges -- reservation + tickets + kilit temizliği aynı transaction'da.
await _reservationWrite.SaveChangesAsync(cancellationToken);

```

backend/.../CreateReservationCommandHandler.cs (gerçek kod)

`_reservationWrite`, `_ticketWrite` ve `_seatLockWrite` üç farklı repository nesnesi ama **aynı** `ApplicationDbContext`'e **bağlılar** (ikisi de `Scoped`, aynı HTTP isteği içinde aynı context'i alıyorlar). Bu yüzden sadece birinden çağrılan `SaveAsync()`, o context'in izlediği **tüm** bekleyen değişiklikleri (3 farklı entity tipini) tek seferde, tek transaction'da veritabanına yazar — Bölüm 6.2'de bunun "DbContext zaten bir Unit of Work'tür" ilkesinin gerçek bir örneği olduğunu göreceğiz.

9) Hata varsa: tek try/catch

Handler'ların hiçbirisi try/catch içermiyor. Zincirin herhangi bir noktasında fırlatılan `NotFoundException`, `ConflictException` veya `ValidationException`, `ExceptionHandlerMiddleware` tarafından yakalanıp tutarlı bir JSON gövdesine ve doğru HTTP koduna çevrilir:

```

catch (NotFoundException ex)
{
    await WriteAsync(context, StatusCodes.Status404NotFound,
        new { title = "Kayıt bulunamadı", status = 404, detail = ex.Message });
}
catch (ConflictException ex)
{
    await WriteAsync(context, StatusCodes.Status409Conflict,
        new { title = "Kaynak çakışması", status = 409, detail = ex.Message });
}

```

backend/Presentation/CineSeat.WebAPI/Middleware/ExceptionHandlerMiddleware.cs (özet, gerçek kod)

Sonuç

17 adımlık bu akışta handler'ın kendisi sadece bir parçadır (~90 satır iş mantığı) — loglama, doğrulama, kimlik doğrulama, hata çevirisi, audit alanları (`CreatedAt`), transaction yönetimi hepsi başka sınıfların işiydi ve otomatik gerçekleşti. İyi mimarinin ölçüsü budur: yeni bir use case yazarken ne kadar az şey düşünmen gerektiği.

Bölüm 6 — Tasarım Deseni Sözlüğü (CineSeat merceğinden)

6.1 Repository Pattern

Literatürde "generic repository" (`IRepository<T>` ile `GetAll`, `Add`, `Update`, `Delete`) sık sık anti-pattern olarak eleştirilir: `IQueryable<T>` döndürmek soyutlamayı anlamsız kılar, çünkü çağırıcı taraf zaten EF Core'a özgü sorgu yazmak zorunda kalır.

CineSeat bu tuzağı kısmen **kendi çözümüyle** aşmış: `IReadRepository<T>` gerçekten `IQueryable<T>` döndürüyor, ama...

- `DbSet<T>` asla dışarı açılmıyor (`protected DbSet<T> Table`) — Application katmanı EF Core'un `DbSet` tipini hiç görmüyor, sadece soyut `IQueryable/Expression` görüyor (bunlar `System.Linq`'e ait, ORM'e değil).
- Sorguyu **materyalize eden** (`ToListAsync`, `CountAsync`...) metotlar `IAsyncQueryExecutor` arkasına gizlenmiş (bkz. 3.2) — Application katmanı EF Core paketine hâlâ bağımlı değil.
- Her entity için 40 metotluk bir "çöplük" yerine, **ince** alt sınıflar var (`CityReadRepository` gibi üç satırlık sınıflar) — generic taban class gerçek işi yapıyor, entity'ye özel kod sadece gerektiğinde eklenir.

Dengeli değerlendirme

CineSeat'in yaklaşımı "aggregate bazlı, spesifik repository" idealiyle "generic repository" arasında bir orta yol: generic taban sınıfların üstüne (gerektiğinde) entity'ye özel metotlar eklenebilecek bir iskelet var, ama bugüne kadar hiçbir alt sınıfta buna ihtiyaç duyulmamış — hepsi hâlâ üç satırlık boş kabuklar. Bu, kötü değil; "aggregate root başına özel bir metoda ihtiyaç doğduğunda ekle" ilkesiyle örtüşüyor.

6.2 Unit of Work

Birden fazla değişikliği tek atomik işlemde toplama deseni. CineSeat'te ayrı bir `IUnitOfWork` arayüzü **yok** — `DbContext` zaten bu görevi görüyor: `WriteRepository<T>.SaveAsync()` doğrudan `_context.SaveChangesAsync()`'e delege ediyor. Bölüm 5'te gördüğümüz gibi, aynı `ApplicationDbContext`'i paylaşan üç farklı repository (`_reservationWrite`, `_ticketWrite`, `_seatLockWrite`), tek bir `SaveAsync()` çağrısıyla hep birlikte kaydediliyor.

```
public async Task<int> SaveAsync(CancellationToken cancellationToken = default)
=> await _context.SaveChangesAsync(cancellationToken);
```

backend/Infrastructure/CineSeat.Persistence/Repositories/WriteRepository.cs (ilgili satır)

6.3 Specification Pattern

Sorgu kriterlerini yeniden kullanılabilir nesnelere dönüştürme deseni. CineSeat kullanmıyor — sorgu ifadeleri (`GetWhere(c => ...)`) doğrudan handler içinde, lambda olarak yazılıyor. 40 civarı handler için bu makul; "aktif film" gibi bir tanım 5 farklı yerde tekrarlanmaya başlarsa (ör. `m.StartDate <= now && m.EndDate >= now` birden fazla query'de tekrar ederse), Specification Pattern gündeme gelebilir.

6.4 Result Pattern

CineSeat'in kod tabanında hem Result hem Result<T> tipleri **tanımlı**:

```
public class Result<T>
{
    public bool IsSuccess { get; }
    public T? Value { get; }
    public string? Error { get; }
    public static Result<T> Success(T value) => new(true, value, null);
    public static Result<T> Failure(string error) => new(false, default, error);
}
```

backend/Core/CineSeat.Application/Common/Models/Result.cs (gerçek kod)

Bulgu — tanımlı ama kullanılmıyor

Kod tabanında hiçbir handler Result<T> döndürmüyor (CreateCinemaCommandHandler düz long, CreateReservationCommandHandler düz ReservationDto döndürüyor). CineSeat hata yönetimini tamamen **exception + ExceptionHandlingMiddleware** üzerinden yapmayı seçmiş — "beklenen" hatalar (kayıt bulunamadı, çakışma) için bile. Bu, orijinal önerinin ("beklenen hatalar için Result, istisnai olanlar için Exception") aksine ama tutarlı ve tek başına geçerli bir tercih: tüm hata yolu tek bir yerde (middleware) toplanıyor, her handler'da if (result.IsFailure) ... tekrarı yok. Result.cs'in kendisi şu an ölü kod — ileride kullanılacaksa ya devreye alınmalı ya da kaldırılmalı.

6.5 Diğer desenler (CineSeat'te görülenler)

Options Pattern

Konfigürasyonu tip güvenli hâle getirir. CineSeat'in JWT ayarları tam olarak bu desenle okunuyor:

```
public class JwtSettings
{
    public const string SectionName = "Jwt";
    public string Issuer { get; set; } = string.Empty;
    public string Audience { get; set; } = string.Empty;
    public string Key { get; set; } = string.Empty;
    public int ExpiryMinutes { get; set; }
}

// Kayıt:
services.Configure<JwtSettings>(configuration.GetSection(JwtSettings.SectionName));
// Kullanım (JwtTokenService içinde):
public JwtTokenService(IOptions<JwtSettings> settings) => _settings = settings.Value;
```

backend/Infrastructure/CineSeat.Infrastructure/Security/JwtSettings.cs + JwtTokenService.cs

Outbox / Saga / Event Sourcing

Üçü de CineSeat'te yok, ve şu anki ölçekte gerekmiyor: tek veritabanı, dağıtık işlem yok, mikroservis yok. Sistem büyür ve rezervasyon tamamlandığında dış bir servise (e-posta, ödeme sağlayıcısı) güvenilir biçimde bildirim göndermek gerekirse, Outbox Pattern (mesajı aynı transaction içinde bir tabloya yazıp ayrı bir arka plan görevinin işlemesi) ilk düşünülecek araç olurdu.

Bölüm 7 — Mimari Yaklaşımlar: CineSeat Neden Onion + CQRS?

Aşağıdaki alternatiflerin hepsi geçerli mimarilerdir; CineSeat'in seçimini bağlamına oturtmak için kısaca karşılaştırıyoruz.

Yaklaşım	Özeti	CineSeat için neden değil / neden bu
N-Tier / Layered	UI → BLL → DAL, bağımlılık yukarıdan aşağı	İş mantığı veri erişimine bağımlı kalırdı — CineSeat'in çok sayıda iş kuralı (rezervasyon, kampanya) için riskli
Hexagonal (Ports & Adapters)	Onion'ın atası, felsefesi aynı	Onion, katmanları .NET dünyasında daha net isimlendiriyor — pratikte fark yok
Clean Architecture	Onion + Hexagonal + Screaming sentezi	CineSeat'in 5 projesi zaten bu sentezi uyguluyor; isim tercihi "Onion"
Vertical Slice	Klasörleme işe göre, katmanlara göre değil	CQRS ile CineSeat zaten yarı yarıya burada — Features/X/Commands Queries dikey dilim mantığına çok yakın
Modular Monolith	Tek deployment, sıkı modül sınırları	CineSeat şu an tek ekip / tek deployment — bu, doğal bir sonraki adım olabilir
Microservices	Her serviste kendi DB, bağımsız deploy	4 kişilik ekip için erken; dağıtık sistem karmaşıklığının faydası yok

Kısacası CineSeat, Onion'ın katman disiplini CQRS'in okuma/yazma ayrımıyla birleştirmiş — 4 kişilik bir ekip, orta ölçekli bir domain (sinema/bilet) için makul bir denge noktası.

Bölüm 8 — CineSeat Proje İskeleti

Gerçek klasör ağacı (özetlenmiş — Features/ altında 15'e yakın modül var, hepsi aynı desende):

```
backend/  
  Core/  
    CineSeat.Domain/  
      Entities/      Movie, Reservation, Ticket, Seat, SeatLock, Cinema,  
                     City, District, Hall, Showtime, Campaign, Comment,  
                     User, Role, Permission, RolePermission, Genre,  
                     MovieGenre, Technology, HallTech, UserFavorite (19 entity)  
      Entities/Common/ BaseEntity (Id, CreatedAt, UpdatedAt, IsDeleted)  
      Enums/          ReservationStatus, TicketType, SeatType, CampaignType, ScreeningFormat  
    CineSeat.Application/  
      Common/  
        Behaviors/    LoggingBehavior, ValidationBehavior  
        Constants/    RoleNames  
    Exceptions/      NotFoundException, ConflictException, UnauthorizedException, ValidationException  
    Interfaces/      IAsyncQueryExecutor, ICurrentUserService, ITokenService, IPasswordHasher  
    Models/          PagedResult<T>, Result<T> (tanımlı, kullanılmıyor -- bkz. 6.4)  
    Features/         Auth, Movies, Cinemas, Cities, Districts, Halls,  
                     Showtimes, Seats, SeatLocks, Tickets, Reservations,  
                     Campaigns, Comments, Genres, MovieGenres, Technologies,  
                     HallTechs, UserFavorites (her biri Commands/ + Queries/ + DTOs/)  
    Repositories/     IReadRepository<T>, IWriteRepository<T> + 20 entity'ye özel çift  
    DependencyInjection.cs  
  Infrastructure/  
    CineSeat.Persistence/  
      Data/           ApplicationDbContext, DbInitializer, EfAsyncQueryExecutor  
      Data/Interceptors/ AuditableEntityInterceptor  
      Repositories/    ReadRepository<T>, WriteRepository<T> + 20 ince alt sınıf çifti  
      Migrations/      InitialCreate, CompleteDatabaseDesign, FixSchemaIssues  
      ServiceRegistration.cs  
    CineSeat.Infrastructure/  
      Security/        JwtSettings, JwtTokenService, Pbkdf2PasswordHasher  
      ServiceRegistration.cs  
  Presentation/  
    CineSeat.WebAPI/  
      Controllers/     18 controller (Auth, Movies, Cinemas, Reservations, ...)  
      Middleware/       ExceptionHandlingMiddleware  
      Services/         CurrentUserService  
      Program.cs
```

backend/ — gerçek proje ağacı (407 .cs dosyası, özetlenmiş)

Referans yönü

```
WebAPI ----> Application ----> Domain  
|               ^  
+----> Persistence -----+  
+----> Infrastructure -----+
```

Domain hiç kimseyi referans almaz. Bu, .csproj dosyalarındaki ProjectReference etiketleriyle DERLEYİCİ tarafından zorlanır -- iyi niyete bırakılmış bir kural değildir.

İleri seviye fikir — mimari testler

Bugün CineSeat'te bu kuralı kod seviyesinde doğrulayan otomatik bir test yok (NetArchTest.Rules gibi bir kütüphaneye "Domain hiçbir dış katmana bağımlı olmamalı" iddiasını CI'da test etmek mümkün). Test projesi kurulduğunda düşünülebilecek ucuz ve değerli bir ilk adım.

Bölüm 9 — Ne Zaman Ne Kullanmalı?

Bu rehberdeki her yapı bir bedel karşılığında gelir: daha fazla dosya, daha uzun oryantasyon, hata ayıklarken daha fazla "bu çağrı nereye gidiyor" sorusu. CineSeat'in kendi ölçeğinde verdiği kararları, genel bir karar tablosuyla karşılaştıralım:

Durum	Öneri	CineSeat nerede duruyor
3 tablolü basit admin paneli	Tek proje, doğrudan DbContext	Genres, Technologies gibi modüller buna çok yakın davranıyor (validator bile yok)
Küçük ama büyüyecek proje	Onion + CQRS + spesifik repository	CineSeat tam olarak burada — 5 katman, ~40 use case, Read/Write repository ayrımı
Karmaşık domain, çok kural	Onion + DDD + rich model + domain event	CineSeat kuralları handler'da topluyor (anemic model) — bir sonraki adım olabilir ama bugün gerekli değil
Yüksek okuma trafiği	Okuma tarafını ayır: projection, cache	IAsyncQueryExecutor + Select projeksiyonu zaten var; cache henüz yok
Çok ekipli büyük sistem	Modular Monolith, gerekirse mikroservis	4 kişilik ekip için gündemde değil

Sağlıklı ilerleme sırası — CineSeat için somut sonraki adımlar

- **Test projesi ekle.** Şu an backend/**/*.Tests altında hiçbir dosya yok — önce handler'lar için birim test (repository'ler mock'lanarak), sonra ApplicationDbContext için entegrasyon testi.
- **CI hattı kur.** dotnet build + dotnet test'i her push'ta otomatik çalıştırmak, bugün elle yapılan doğrulamayı otomatikleştir (bkz. 2.7).
- **Soft-delete'i tamamla.** IsDeleted altyapısı hazır; WriteRepository.Remove()'un gerçekten IsDeleted = true set etmesi (ya da bilinçli olarak "biz hard delete istiyoruz" kararının netleştirilmesi) küçük ama önemli bir tutarlılık adımı (bkz. 2.6).
- **Result<T> için karar ver.** Ya tamamen kaldır (ölü kod), ya da gerçekten kullanmaya başla — ikisinin bir arada, kullanılmadan durması kod okuyan yeni bir katılımcıyı yanıltabilir (bkz. 6.4).

Kilit ayırım

Karmaşıklığı ihtiyaç doğmadan ekleme, ama sınırları en baştan koy. CineSeat bunu büyük ölçüde doğru yapmış: katman sınırları (Onion) ve sorumluluk ayrımı (CQRS) en baştan kurulmuş; eksik kalan parçalar (test, CI, soft-delete tamamlama) sonradan eklenmesi ucuz olan, "acı gelince ekle" kategorisindeki şeyler.

Bölüm 10 (Bonus) — Frontend Entegrasyonu: React ↔ ASP.NET Core

Orijinal rehberde olmayan, ama CineSeat'in gerçek hikâyesinin bir parçası olan bir bölüm: React (Vite, localhost:5173) ile ASP.NET Core Web API'nin (localhost:5207) birbirine bağlanması sırasında yaşanan gerçek sorunlar ve çözümleri.

10.1 CORS: farklı origin, farklı dünya

Tarayıcı, bir sayfanın farklı bir origin'e (host+port farklıysa) istek atmasını varsayılan olarak engeller — bu bir güvenlik önlemidir (Same-Origin Policy). Backend açıkça izin vermezse frontend hiçbir API çağrısı yapamaz, token doğru olsa bile:

```
const string FrontendCorsPolicy = "FrontendCorsPolicy";
builder.Services.AddCors(options =>
{
    options.AddPolicy(FrontendCorsPolicy, policy =>
    {
        policy.WithOrigins("http://localhost:5173", "http://127.0.0.1:5173")
            .AllowAnyHeader()
            .AllowAnyMethod();
    });
});

// CORS, kimlik doğrulamadan ÖNCE gelmeli -- tarayıcı asıl isteği göndermeden
// önce bir "preflight" (OPTIONS) isteği atar, bu istek hiç token taşımaz.
app.UseCors(FrontendCorsPolicy);
app.UseAuthentication();
app.UseAuthorization();
```

backend/Presentation/CineSeat.WebAPI/Program.cs (gerçek kod)

Yaşanan gerçek sorun — HTTPS yönlendirme tuzağı

CORS doğru kurulduktan sonra bile tarayıcıda Sunucuya ulaşamıyor hatası devam etti. Kök neden CORS değildi: Program.cs'teki (projenin şablonundan gelen, hazır) `app.UseHttpsRedirection();` satırı, `http://localhost:5207` isteklerini otomatik olarak `https://localhost:7085`'e **307 yönlendirmesiyle** gönderiyordu. Tarayıcı bu adresteki güvenilmeyen (self-signed) geliştirme sertifikasını reddediyor, `fetch()` de bunu CORS hatasına çok benzeyen ama aslında tamamen farklı bir ağ hatası olarak gösteriyordu. Çözüm: `if (!app.Environment.IsDevelopment()) { app.UseHttpsRedirection(); }` — yönlendirme sadece üretimde devrede, geliştirmede devre dışı.

10.2 apiClient.js — merkezi fetch katmanı

Frontend tarafında tüm servisler tek bir noktadan geçiyor: base URL, Authorization header'ı ve backend hata gövdesinin frontend hata sınıflarına çevirisi burada, tek yerde:

```

const API_BASE_URL = import.meta.env.VITE_API_BASE_URL || "http://localhost:5207/api";

async function apiRequest(path, { method = "GET", body, headers } = {}) {
  const token = getStoredToken(); // sessionStorage['cineseat_user'].token

  const response = await fetch(`${API_BASE_URL}${path}`, {
    method,
    headers: {
      ...(body !== undefined ? { "Content-Type": "application/json" } : {}),
      ...(token ? { Authorization: `Bearer ${token}` } : {}),
      ...headers,
    },
    body: body !== undefined ? JSON.stringify(body) : undefined,
  }).catch(() => {
    throw new ApiError("Sunucuya ulaşılamıyor. Backend çalışıyor mu kontrol edin.", { status: 0 });
  });

  if (!response.ok) throw await toApiError(response);
  if (response.status === 204) return null;
  return response.json();
}

```

frontend/src/services/apiClient.js (gerçek kod, kısaltılmış)

`toApiError`, backend'in `ExceptionHandlerMiddleware`'inin ürettiği `{ title, status, detail } / { title, status, errors }` gövdesini okuyup HTTP durum koduna göre frontend'in zaten var olan hata sınıflarına (`ValidationError`, `NotFoundError`, `ConflictError`...) çeviriyor — backend ile frontend'in hata sözleşmesi bu tek dosyada buluşuyor.

10.3 Rol eşleme: Admin/User ↔ admin/member

Backend rolleri `RoleNames.Admin / RoleNames.User` (büyük harfle başlıyor, bkz. Bölüm 3.2), frontend'in `ProtectedRoute` bileşeni ise `allowedRoles=[ "admin" ]` gibi küçük harfli string bekliyor. `authService.js` içindeki `mapRole()` bu ikisini köprülüyor — küçük ama gözden kaçırılması kolay bir uyumsuzluk noktası.

Ek A — Terimler Sözlüğü

Terim	Kısa açıklama
Aggregate	Birlikte değişen, birlikte tutarlı olması gereken entity grubu (ör. Reservation + Ticket)
Anemic Model	Sadece veri tutan, davranışı olmayan entity — CineSeat'in tüm entity'leri böyle
Assembly	Derlenmiş .NET projesinin çıktısı — CineSeat'te 5 tane (.dll/.exe)
Attribute	Sınıf/metot üstüne yapıştırılan etiket: [HttpGet], [Authorize]
CLR	.NET kodunu çalıştıran motor (Common Language Runtime)
Command	Durumu değiştiren mesaj (ör. CreateReservationCommand)
Composition Root	Bağımlılıkların bir araya getirildiği tek nokta — CineSeat'te Program.cs
CQRS	Okuma ve yazma modellerinin ayrılması
CRUD	Create, Read, Update, Delete
DIP	Bağımlılığın tersine çevrilmesi prensibi — arayüz içeride, implementasyon dışarıda
DTO	Sadece veri taşıyan, davranışsız nesne (ör. MovieDto, AuthUserDto)
Entity	Kimliği (Id) olan, zaman içinde değişen nesne
Global Query Filter	Her sorguya otomatik eklenen WHERE koşulu — CineSeat'te IsDeleted == false
Handler	Tek bir use case'i yürüten sınıf (ör. CreateCinemaCommandHandler)
IDOR	Insecure Direct Object Reference — kullanıcı kimliğinin istekten değil, token'dan okunmasıyla önlenir
Interceptor	SaveChanges öncesi/sonrası devreye giren EF Core kancası (ör. AuditableEntityInterceptor)
JWT	İmzalı, kendi kendini doğrulayan kimlik token'ı (CineSeat: HS256)
Mediator	Nesneler arası iletişimi merkezileştiren aracı (CineSeat: MediatR)
Migration	Veritabanı şema değişikliğini tarif eden dosya
ORM	Nesne ile veritabanı arasında tercümanlık yapan kütüphane (CineSeat: EF Core + Npgsql)
PBKDF2	Parolayı geri döndürülemez biçimde "eritmek" için kullanılan, tuzlu ve tekrarlı bir özet (hash) algoritması
Pipeline Behavior	Handler'ın etrafını saran, isteği/cevabı gözlemleyen/değiştiren MediatR halkası
POCO	Hiçbir kütüphaneye bağımlı olmayan sade C# sınıfı
Projection	Sorguda sadece gereken kolonların Select ile seçilmesi
Query	Veri okuyan, durumu değiştirmeyen mesaj
Repository	Veri erişimini soyutlayan katman
Result Pattern	Hatayı exception yerine dönüş değeriyle taşıma deseni — CineSeat'te tanımlı ama kullanılmıyor
Soft Delete	Kayı fiziksel silmeyi "silindi" işaretlemek — CineSeat'te altyapı hazır, write path henüz bağlı değil
Unit of Work	Birden fazla değişikliği tek atomik işlemde kaydetme — CineSeat'te DbContext'in kendisi
Use Case	Sistemin sunduğu tek bir anlamlı iş (ör. "Rezervasyon oluştur")
Validator	Verinin şeklen doğruluğunu kontrol eden sınıf (FluentValidation)

Ek B — CineSeat Kod Tabanından Gözlemler

Bu rehberi hazırlarken kod tabanının tamamı (407 C# dosyası) tarandı. Aşağıdaki liste hem iyi uygulanan noktaları hem de somut iyileştirme fırsatlarını, dosya referanslarıyla birlikte özetliyor.

İyi uygulanmış noktalar

- **IDOR koruması tutarlı.** `CreateReservationCommandHandler`, `AddCommentCommandHandler` gibi tüm yazma işlemlerinde kullanıcı kimliği JWT'den okunuyor, hiçbir `Command/Query` `UserId` alanı istekten kabul etmiyor.
- **AsNoTracking disiplini.** Tüm query handler'lar okuma sorgularında tutarlı biçimde `tracking: false` geçiyor — gereksiz change-tracking maliyeti yok.
- **IAsyncQueryExecutor ile EF Core sızıntısı önlenmiş.** `Application` katmanı, `Microsoft.EntityFrameworkCore` paketini hiç referans almadan asenkron sorgu çalıştırabiliyor (bkz. 3.2) — literatürdeki DIP örneklerinden daha ileri bir soyutlama.
- **RoleNames sabitleri tek kaynak.** `Seed` verisi, `RegisterCommandHandler` ve `[Authorize(Roles = ...)]` öznitelikleri aynı sabitleri paylaşıyor — yazım hatası riski yok.
- **Aynı hata mesajı, kullanıcı yok / parola yanlış için.** `LoginCommandHandler` ikisini de aynı `UnauthorizedException` mesajıyla döndürüyor — hangi e-postaların kayıtlı olduğunu sızdırmıyor.
- **Kampanya indirimini taşıma koruması.** `Math.Min(raw, subtotal)` — indirim asla ara toplamı aşmıyor, `Total` negatife düşemiyor.

Somut iyileştirme fırsatları

- **Soft-delete tamamlanmamış.** `IsDeleted` alanı ve global query filter okuma tarafında hazır, ama `WriteRepository<T>.Remove()` hâlâ gerçek DELETE üretiyor — kod tabanında `IsDeleted = true` atayan tek satır yok (bkz. 2.6).
- **Result<T> tanımlı ama kullanılmıyor.** `Common/Models/Result.cs` hiçbir handler'da referans edilmiyor — ölü kod ya da netleştirilmemiş bir plan (bkz. 6.4).
- **Test projesi yok.** `backend/**/Tests` altında hiç dosya yok; CI/CD hattı da kurulmamış (bkz. 2.7, Bölüm 9).
- **Command/Query'ler record değil.** `Mutable class + property setter` kullanılıyor; bugüne kadar sorun çıkarmadı ama pipeline büyüdükçe (ör. yeni bir behavior eklenince) teorik bir risk taşıyor (bkz. 4.5).
- **Fluent API tek dosyada.** 19 entity için `OnModelCreating` hâlâ yönetilebilir boyutta; entity sayısı arttıkça `IEntityTypeConfiguration<T>` dosyalarına bölünmesi düşünülebilir (bkz. 2.3).
- **Bazı modüllerde validator yok.** `Cities` gibi basit CRUD modüllerinde `ValidationBehavior` sessizce devre dışı kalıyor (`_validators.Any()` false) — bilinçli bir sadeleştirme, ama büyüyen bir modülde gözden kaçmaması gerekir.

Not

Bu liste bir eleştiri değil, bir anlık fotoğraf. Amacı, ekibin "şu an nerede durduğumuzu" tartışırken somut, kod tabanlı bir başlangıç noktasına sahip olmasıdır.